

SR_leak_weights_viz_fast

January 10, 2026

Weight Distribution Analysis for Leak Parameter Sweep

This script visualizes how (W) and inhibitory (A) weight distributions change as a function of the leak parameter (1/r) in Continuous Hopfield Networks.

Key quantities: - **weights W_{ij}**: Learned synaptic connections between neurons - **Inhibitory weights A_i**: Self-inhibition accumulated during sleep retrieval, reconstructed as $A_i = \beta * \text{sum}(\text{retrieved_patterns})$

Simulation parameters: - $\rho = 0.5$ (pattern correlation) - $\beta = 0.1$ (inhibitory plasticity rate)

Data sources: - SR_leak_sweep: leak values [0.5, 1.0, 1.5, 2.0] - SR_leak_strong_sweep: leak values [2.0, 3.0, 4.0, 5.0, 6.0]

```
[ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import json
from pathlib import Path
import sys

sys.path.insert(0, str(Path(__file__).parent.parent))
from utils import DATA_DIR, plot_density_3d_bw

PLOTS_DIR = Path(__file__).parent.parent / "plots"
PLOTS_DIR.mkdir(exist_ok=True)

[ ]: plt.rcParams.update({
    'text.usetex': True, 'font.family': 'serif', 'font.serif': ['Times'],
    'font.size': 20, 'font.weight': 'bold',
    'axes.labelsize': 25, 'axes.titlesize': 25, 'axes.linewidth': 1.5, 'axes.
    ↳grid': False,
    'xtick.labelsize': 20, 'ytick.labelsize': 20, 'lines.linewidth': 2.5,
})

def style_ax(ax):
    for s in ['bottom', 'left']: ax.spines[s].set_linewidth(1.5)
    for s in ['top', 'right']: ax.spines[s].set_visible(False)
    ax.tick_params(bottom=True, left=True, top=False, right=False)
```

Configuration

```
[ ]: TARGET_NS, TARGET_NP = 250, 8
DENSITY_LEAKS = [1.0, 2.0, 4.0, 6.0] # For 3D density plots
ALL_LEAKS = [0.5, 1.0, 1.5, 2.0, 3.0, 4.0, 5.0, 6.0] # For comparison plot
```

Data Loading

```
[ ]: def get_index(data_dir):
    """Load or build simulation index (cached to JSON)."""
    index_file = data_dir / "sim_index.json"
    if index_file.exists():
        raw = json.load(open(index_file))
        return {eval(k): [data_dir / d for d in v] for k, v in raw.items()}
    print(f" Building index for {data_dir.name}...")
    index = {}
    for sim_dir in data_dir.glob("sim_nb_*"):
        p = dict(line.split('=', 1) for line in (sim_dir / "parameters.data").
        ↪read_text().strip().split('\n') if '=' in line)
        key = (float(p['leak']), int(p['network_size']), int(p['num_patterns']))
        index.setdefault(key, []).append(sim_dir.name)
    json.dump({str(k): v for k, v in index.items()}, open(index_file, 'w'))
    return {k: [data_dir / d for d in v] for k, v in index.items()}

def load_inhib(sleep_dir):
    """Load inhibitory weights from a sleep simulation."""
    p = dict(line.split('=', 1) for line in (sleep_dir / "parameters.data").
    ↪read_text().strip().split('\n') if '=' in line)
    patterns = np.loadtxt(sleep_dir / "patterns.data")
    results = pd.read_csv(sleep_dir / "results.data")
    valid_idx =
    ↪results['recovered_pattern_idx'][results['recovered_pattern_idx'] >= 0]
    return float(p.get('beta', 0.1)) * patterns[valid_idx].sum(axis=0)

[ ]: print("Loading indices...")
write_index = get_index(DATA_DIR / "trained_networks" / "SR_leak_sweep")
sleep_index = get_index(DATA_DIR / "sleep_results" / "SR_leak_sleep")
write_index_strong = get_index(DATA_DIR / "trained_networks" /
    ↪"SR_leak_strong_sweep")
sleep_index_strong = get_index(DATA_DIR / "sleep_results" /
    ↪"SR_leak_strong_sleep")

# Merge indices (strong takes precedence for overlapping leaks)
def merge_indices(idx1, idx2):
    merged = dict(idx1)
    merged.update(idx2)
    return merged
```

```

write_index_all = merge_indices(write_index, write_index_strong)
sleep_index_all = merge_indices(sleep_index, sleep_index_strong)

print("Loading weights and inhibition...")
weights_by_leak, inhib_by_leak = {}, {}
weights_lists, inhib_lists = {}, {}

for leak in ALL_LEAKS:
    key = (leak, TARGET_NS, TARGET_NP)
    if key not in write_index_all:
        print(f" Warning: {key} not found, skipping")
        continue
    # Main
    all_w = [np.loadtxt(d / "weights.data").flatten() for d in write_index_all[key]]
    weights_by_leak[leak] = np.concatenate(all_w)
    weights_lists[leak] = all_w
    # Inhibitory
    if key in sleep_index_all:
        all_i = [load_inhib(d) for d in sleep_index_all[key]]
        inhib_by_leak[leak] = np.concatenate(all_i)
        inhib_lists[leak] = all_i
    print(f" leak={leak}: {len(all_w)} reps")

```

Loading indices...

Loading weights and inhibition...

```

leak=0.5: 2 reps
leak=1.0: 2 reps
leak=1.5: 2 reps
leak=2.0: 2 reps
leak=3.0: 2 reps
leak=4.0: 2 reps
leak=5.0: 2 reps
leak=6.0: 2 reps

```

3D Density Plots (leak 1, 2, 4, 6)

```

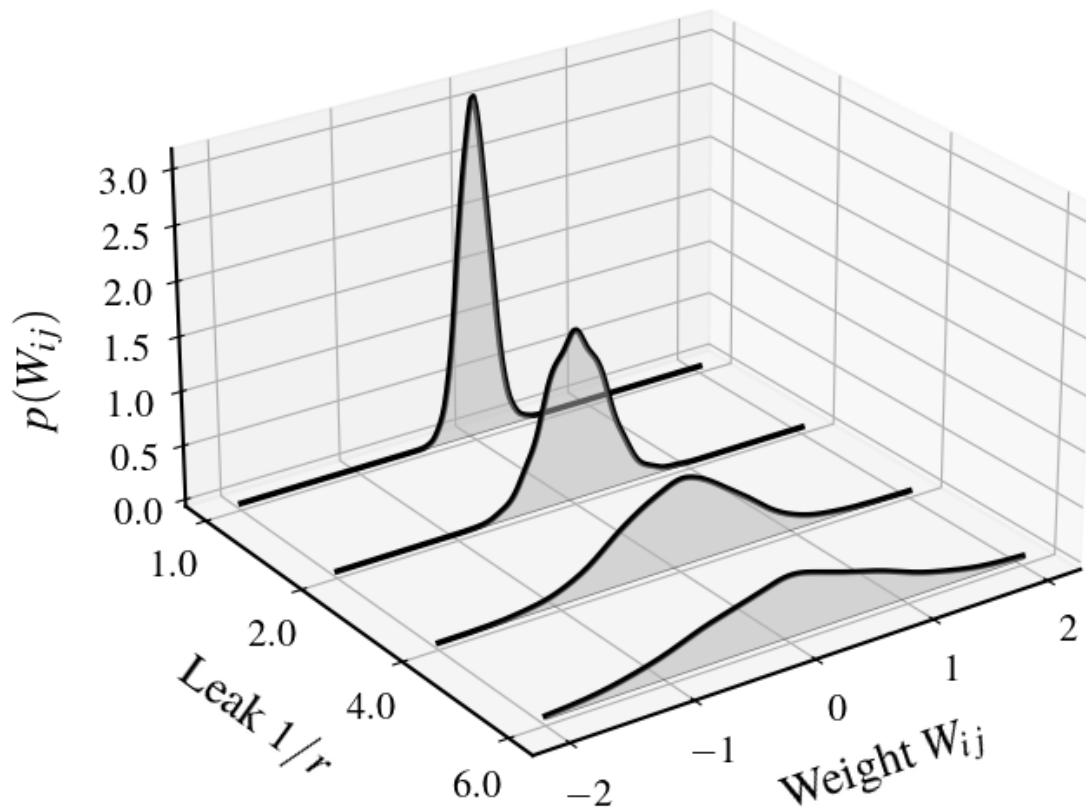
[ ]: print("\nPlotting main/storage weights 3D density...")
weights_density = {l: weights_by_leak[l] for l in DENSITY_LEAKS if l in weights_by_leak}
fig, ax = plot_density_3d_bw(weights_density, xlabel=r'Weight $W_{ij}$',
    ylabel=r'Leak $1/r$', zlabel=r'$p(W_{ij})$',
    depth_spacing=1.2, alpha=0.5, bandwidth=0.2,
    view_elev=25, view_azim=55, figsize=(10, 7),
    xlim=(-2, 2))
ax.set_title(r'Main/storage Weight Distribution', fontsize=18, pad=10)

```

```
plt.savefig(PLOTS_DIR / 'SR_leak_weights_density_3d.png', dpi=300,
            ↳bbox_inches='tight')
print("Saved SR_leak_weights_density_3d.png")
plt.show()
```

Plotting main/storage weights 3D density...
 Saved SR_leak_weights_density_3d.png

Main/storage Weight Distribution



```
[ ]: print("Plotting inhibitory weights 3D density...")
inhib_density = {l: inhib_by_leak[l] for l in DENSITY_LEAKS if l in
↳inhib_by_leak}
fig, ax = plot_density_3d_bw(inhib_density, xlabel=r'Inhibition $A_i$',
↳ylabel=r'Leak $1/r$', zlabel=r'$p(A_i)$',
```

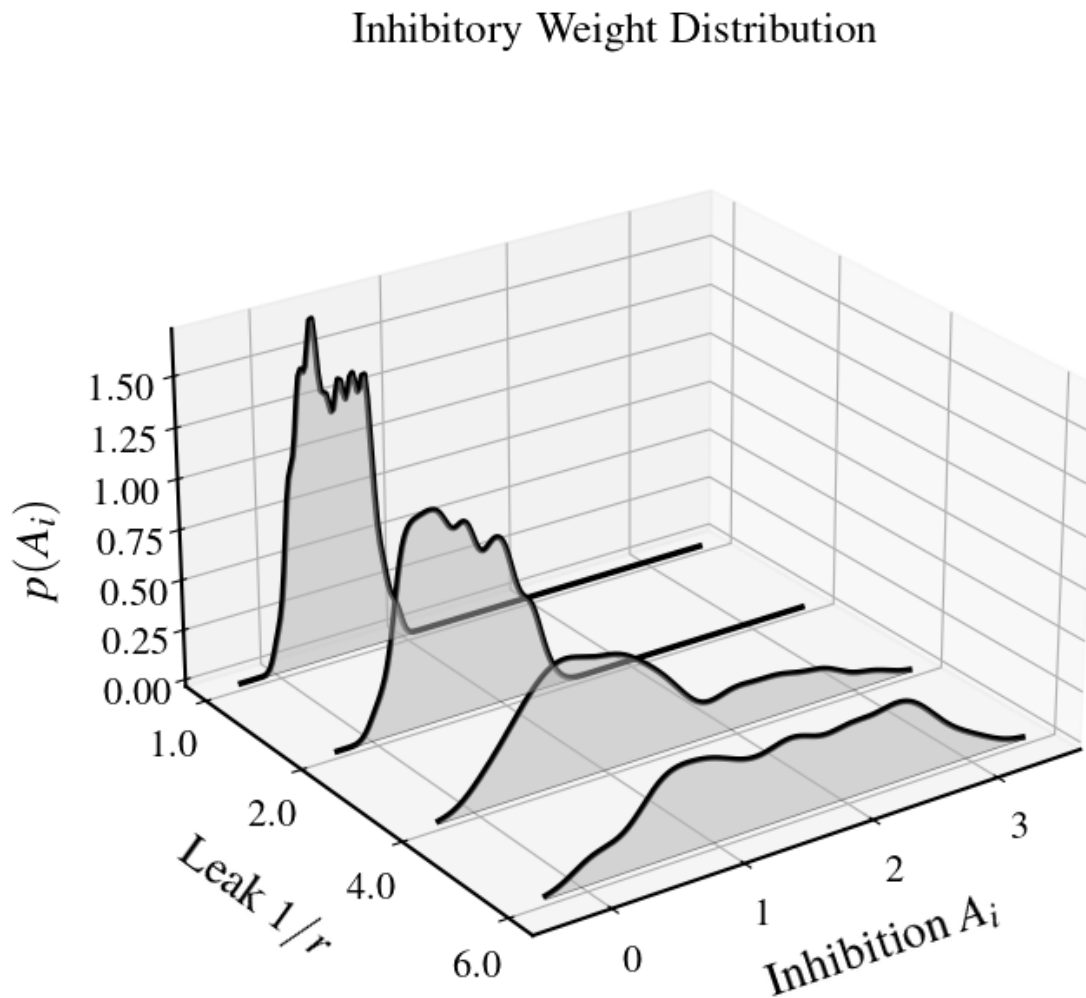
```

depth_spacing=1.2, alpha=0.5, bandwidth=0.2,
view_elev=25, view_azim=55, figsize=(10, 7))
ax.set_title(r'Inhibitory Weight Distribution', fontsize=18, pad=10)
plt.savefig(PLOTS_DIR / 'SR_leak_inhib_density_3d.png', dpi=300,
bbox_inches='tight')
print("Saved SR_leak_inhib_density_3d.png")
plt.show()

```

Plotting inhibitory weights 3D density...

Saved SR_leak_inhib_density_3d.png



Extended Comparison (all leak values)

```

[ ]: print("\nPlotting extended comparison...")
leaks = sorted(weights_lists.keys())

```

```

avg_main = [np.mean([np.mean(np.abs(x)) for x in weights_lists[l]]) for l in leaks]
avg_inhib = [np.mean([np.mean(np.abs(x)) for x in inhib_lists[l]]) if l in inhib_lists else np.nan for l in leaks]

fig, ax = plt.subplots(figsize=(8, 6))
ax.plot(leaks, avg_main, 'o-', color='black', linewidth=2, markersize=10,
        markerfacecolor='white', markeredgecolor='black', markeredgewidth=2,
        label=r'$\langle |W_{ij}| \rangle$')
ax.plot(leaks, avg_inhib, 's-', color='black', linewidth=2, markersize=10,
        markerfacecolor='black', label=r'$\langle A_i \rangle$')
ax.set_xlabel(r'Leak $1/r$')
ax.set_ylabel(r'Average absolute weight', labelpad=15)
ax.legend(frameon=False)
style_ax(ax)
plt.tight_layout()
plt.savefig(PLOTS_DIR / 'SR_leak_weights_comparison_extended.png', dpi=300,
            bbox_inches='tight')
print("Saved SR_leak_weights_comparison_extended.png")
plt.show()

```

Plotting extended comparison...

Saved SR_leak_weights_comparison_extended.png

